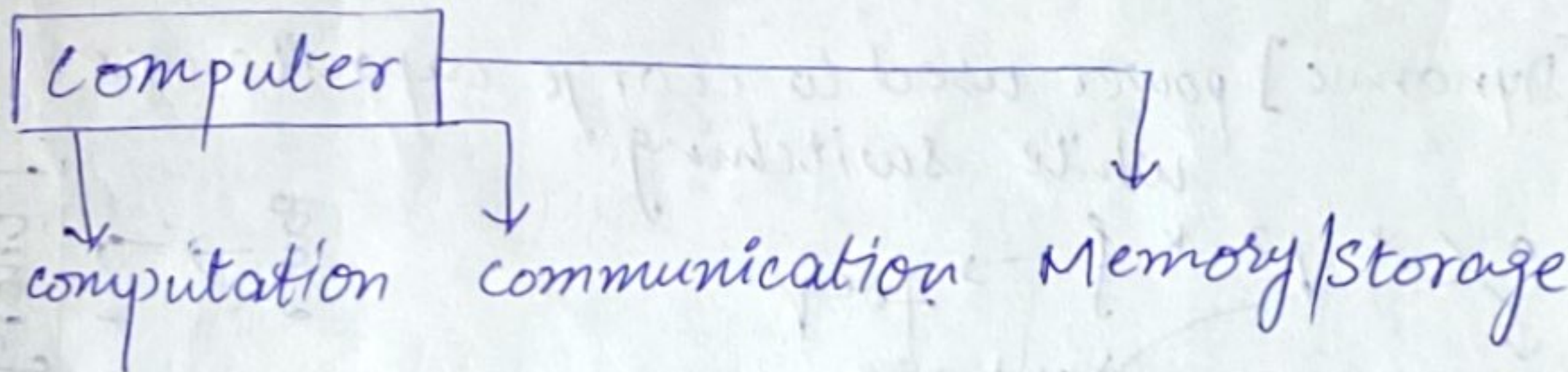
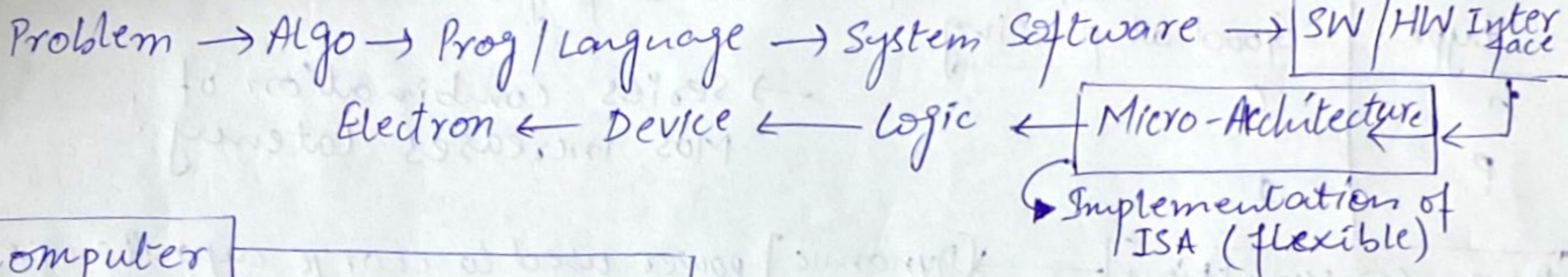


How Computer solves Problem?

→ By making electrons play as per the requirement:

→ Path:



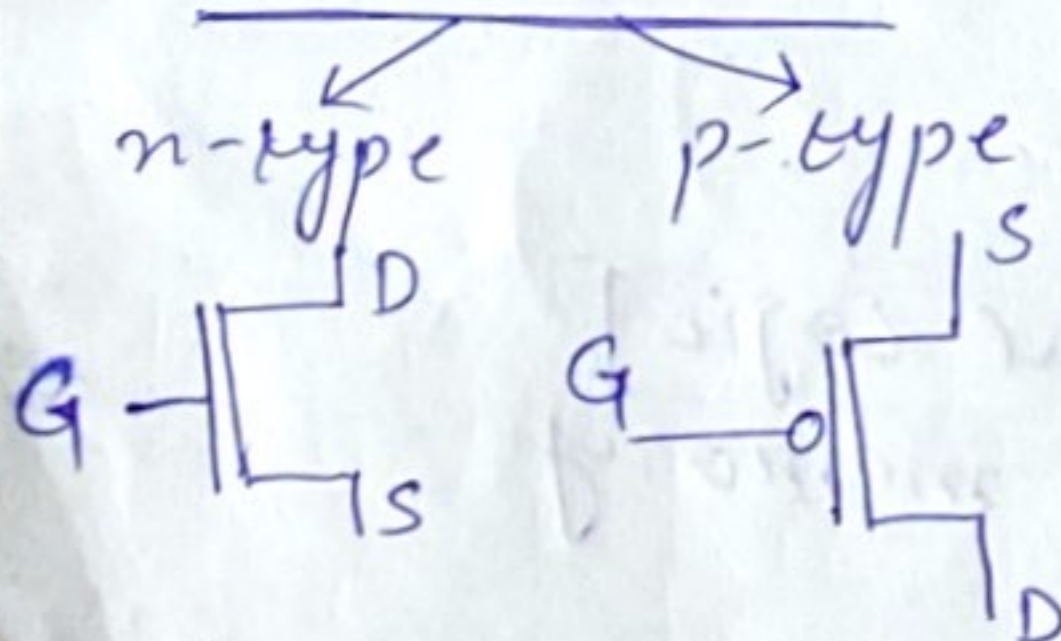
General Purpose Sys. Vs. Special Purpose Sys.

→ Flexible  
→ Easier to handle  
→ Not best for performance and efficiency  
→ e.g. CPU

→ Not flexible  
→ Difficult to handle  
→ Efficient for application  
→ e.g. ASICs

Transistors: used for making logic gates

→ MOS Transistor



Used as a switch here!

n-type

- $G = 3V$  (High) → Switch closed
- open the other way
- Good at pulling down

p-type

- $G = 0V$  (Low) → Switch closed
- open the other way
- Good at pulling up

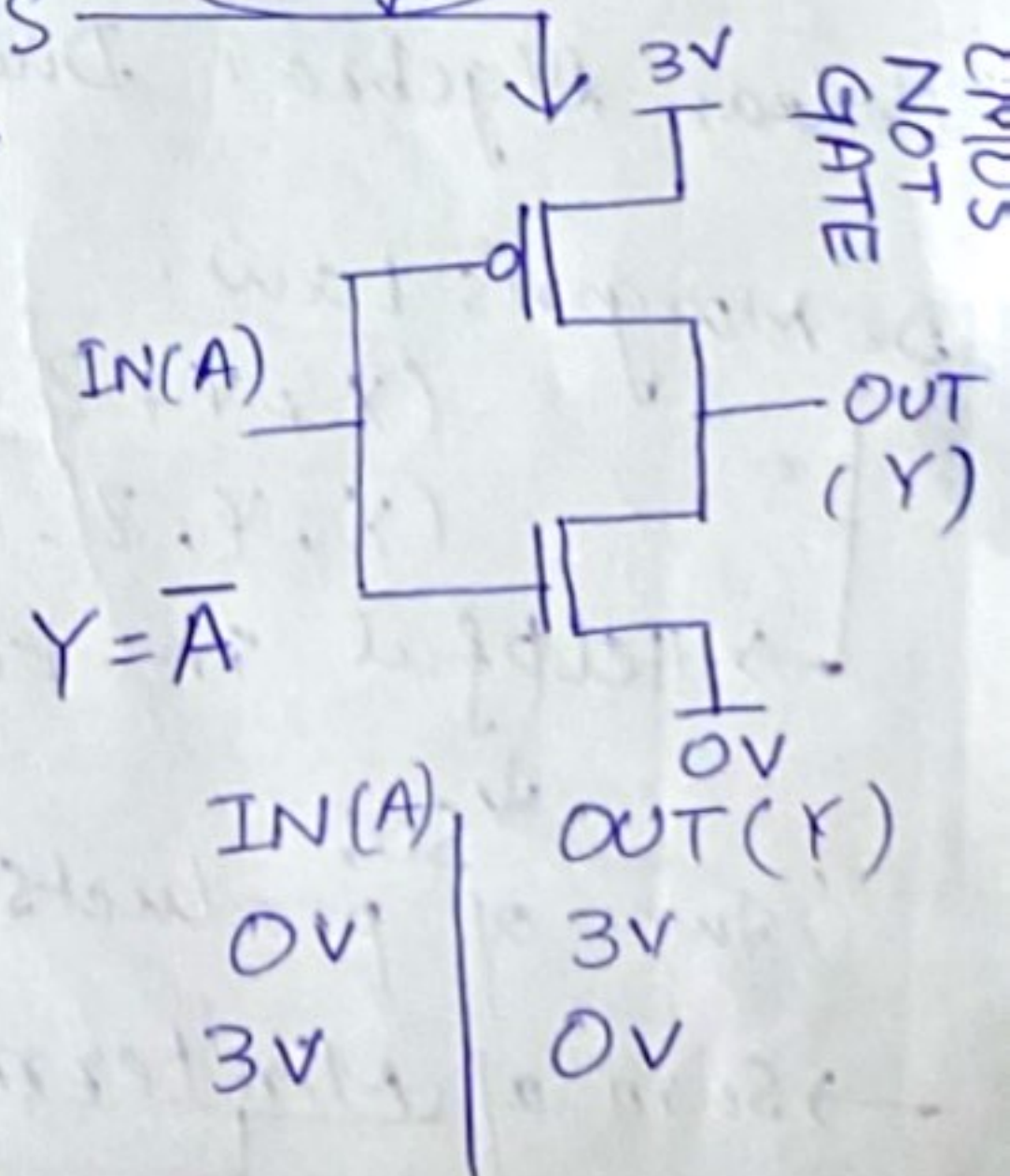
Logic Gates:

→ we will be making gates using the CMOS

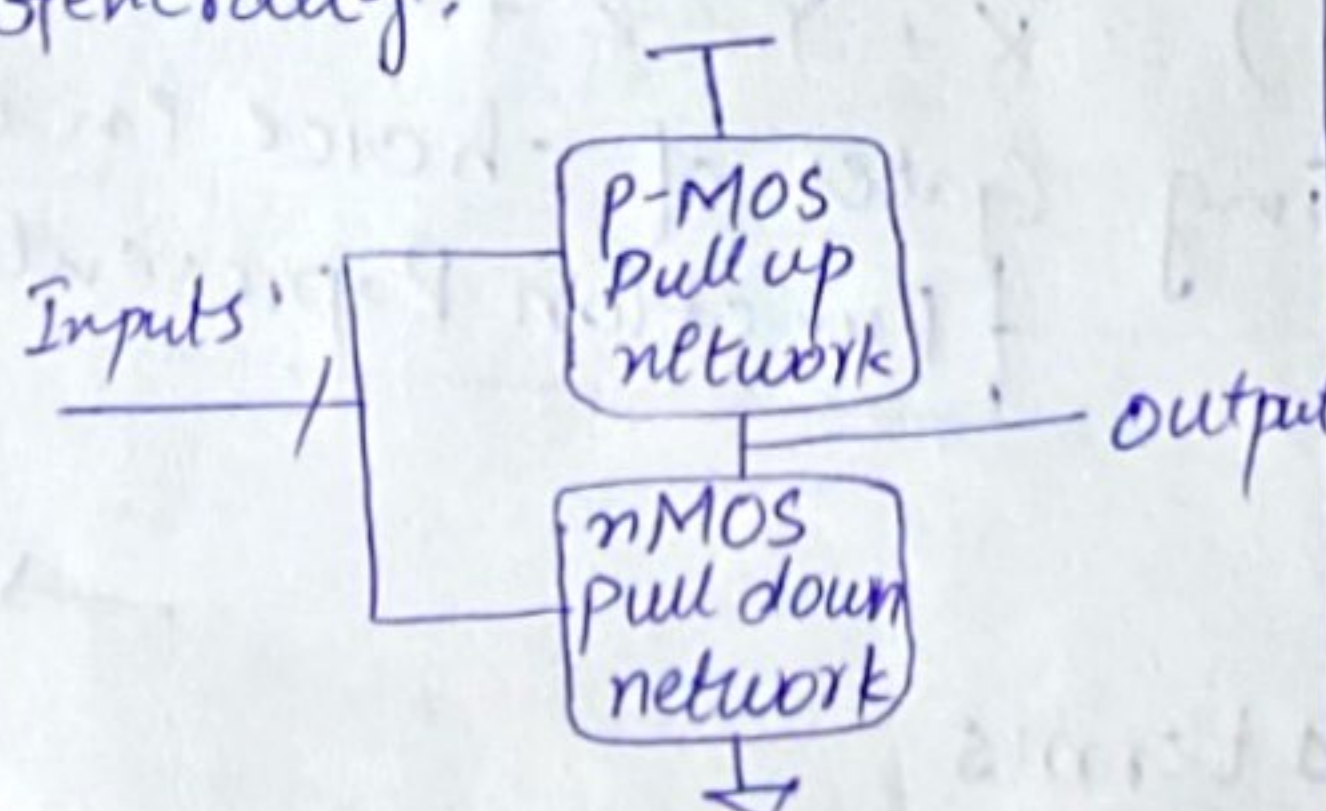
e.g. AND Gate  
NAND Gate  
NOT Gate

nMOS + pMOS = CMOS

e.g.



Generally:



Transistors

• Parallel

→ ON if one transistor is ON

• Series

→ ON if all transistors are ON



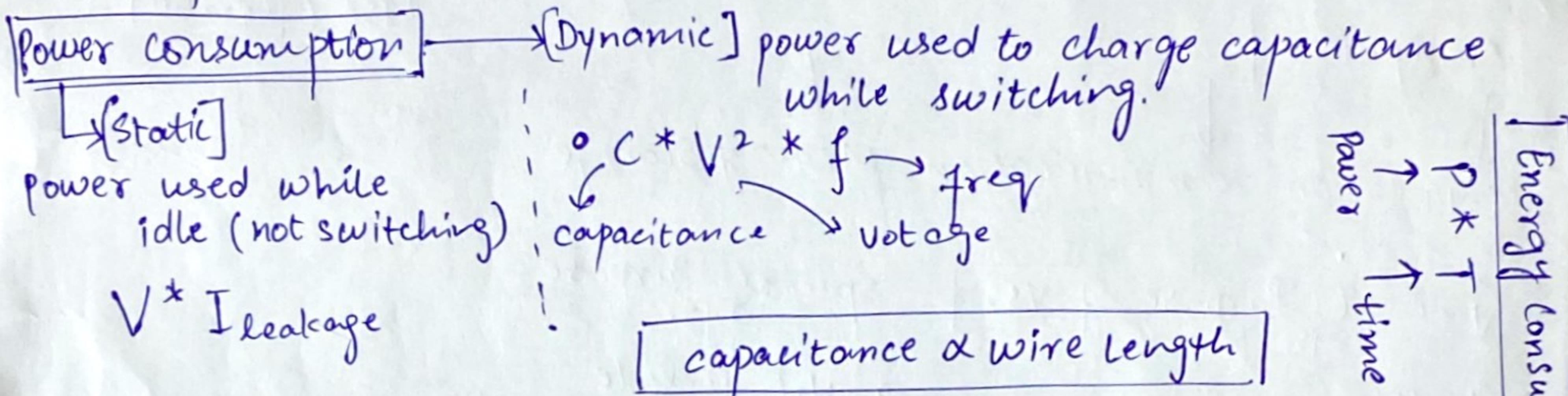
## Underlying operational characteristics of CMOS:

n-type → Bad at pulling up, p-type → Bad at pulling down

Reason: Goes down to physics.

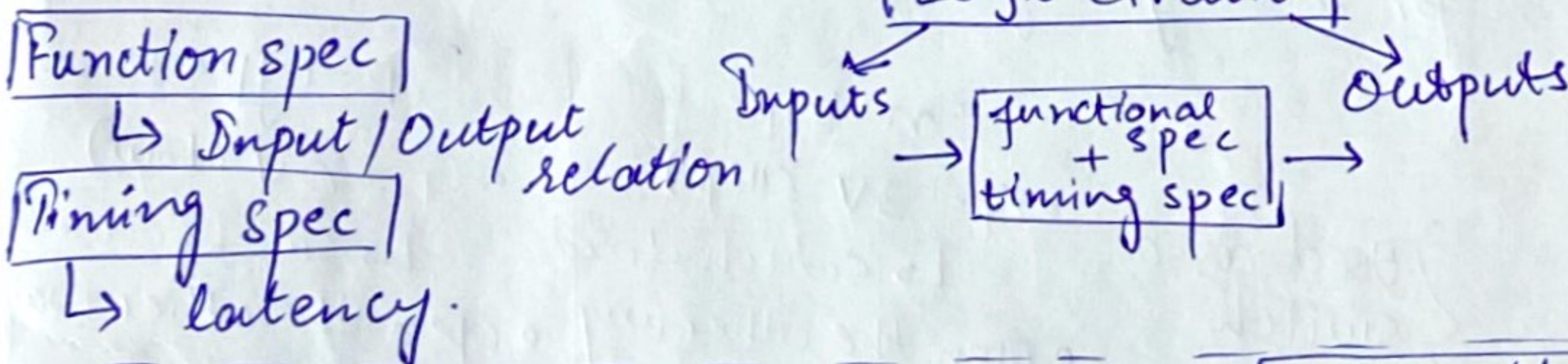
| type: | Good at passing |
|-------|-----------------|
| n     | 0               |
| p     | 1               |

→ Series combination of MOS increases latency!



**MOORE'S LAW** • No # transistor increases, keeping area constant, as time goes on, implying costs of transistors decreases.

## How we build Logic Circuits?



type ① **Combinational logic**  
↳ memory less

type ② **sequential logic**  
↳ Has memory

## Boolean Algebra: Duality

AND  $\xleftrightarrow{\text{Dual}}$  OR  
1  $\xleftrightarrow{\text{Dual}}$  0

literal and complement won't change!

## De Morgan's Law:

$$(X + Y + Z + \dots) = \overline{X} \cdot \overline{Y} \cdot \overline{Z} \cdot \dots$$

$$(X \cdot Y \cdot Z \cdot \dots) = \overline{X} + \overline{Y} + \overline{Z} + \dots$$

→ Helpful in keeping Gates of choice in design.

↓ SOP  
(sum of products)

**Function Representation**

↓ POS  
[product of sum]

→ Sum of Min terms  
eval → 1 terms

→ Product of Max terms  
eval → 0 terms



L#2 P#2

③

25/04/2026

→ why we simplify to reduce cost and latency!

26/04/2026

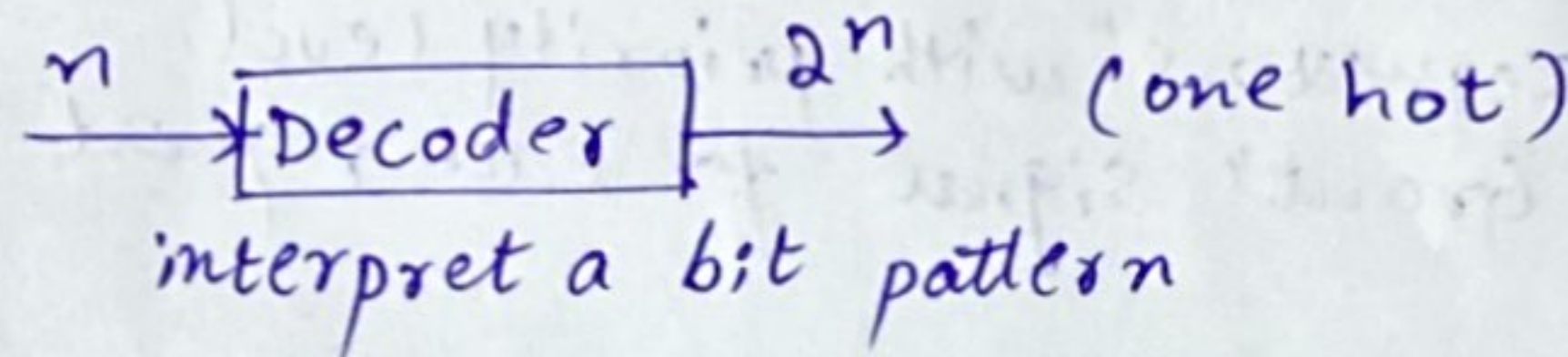
→ Min term  $\leftrightarrow$  Max term Conversion

e.g.  $\sum m(1, 3, 5) \Leftrightarrow \prod M(0, 2, 4)$

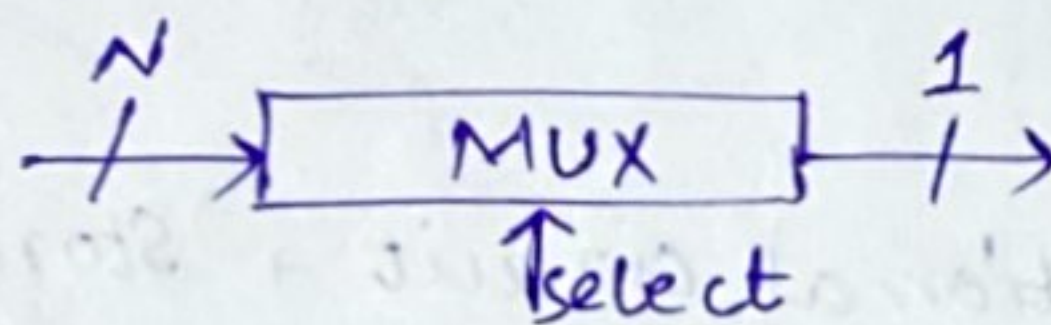
Combinational Blocks Group of combinational circuits.

↓ examples

① Decoder:

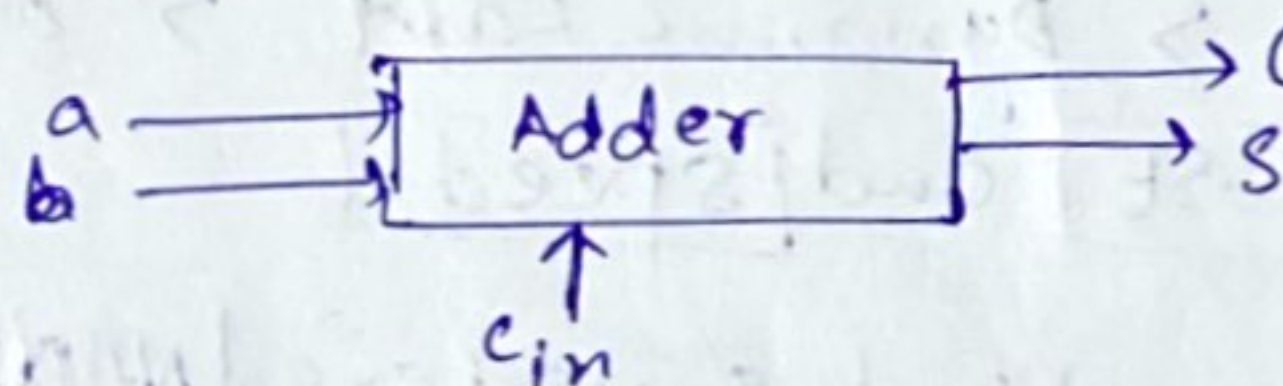


② Multiplexer:



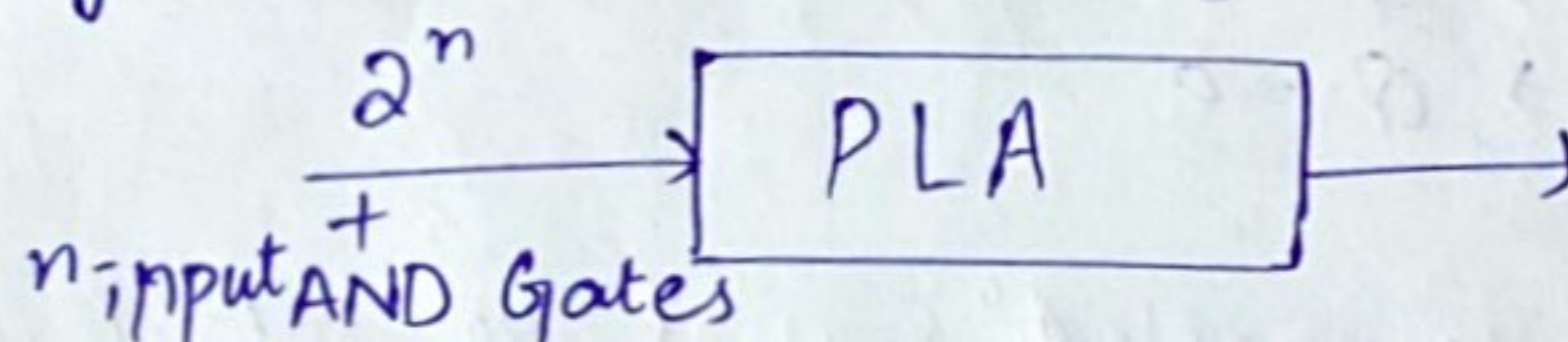
select an input based on the select signal.

③ Full Adder:



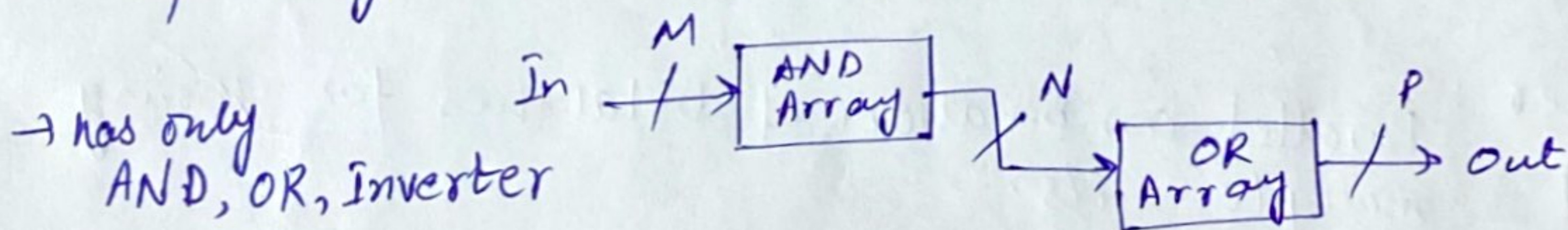
• performs 1 bit addition while taking  $C_{in}$  and giving  $C_{out}$ .  
• Series combo gives  $N$ -bit adder (Increase latency)

④ Programmable Logic Array:



• We program the connection

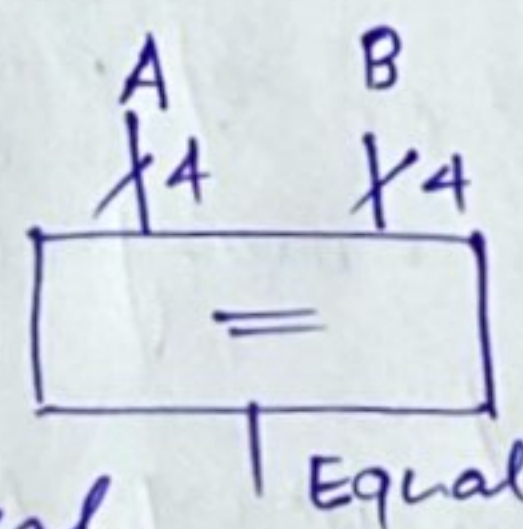
connect the output of an AND Gate to the input of an OR Gate if corresponding minterm is included in the SOP.



→ has only AND, OR, Inverter

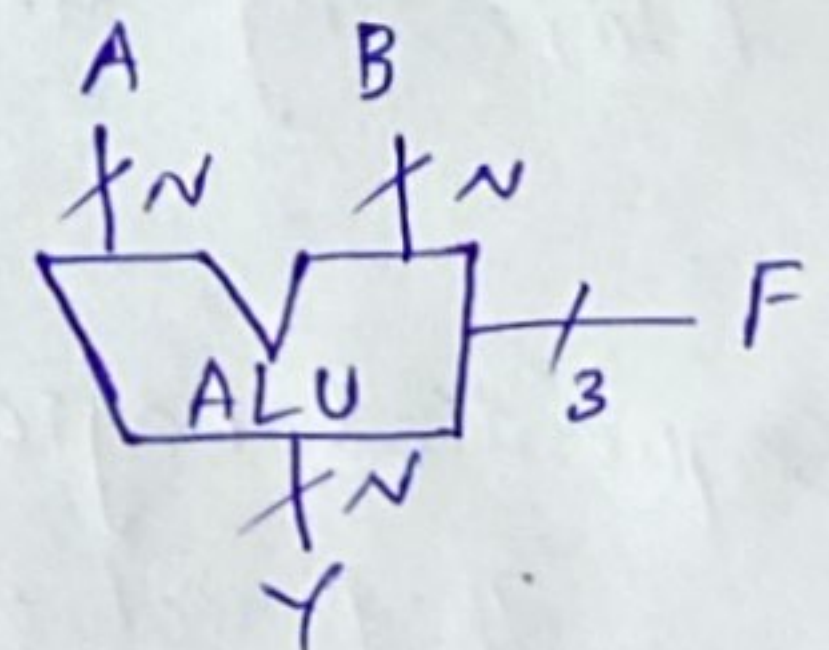
⑤ Comparators:

case: equality checker: e.g. 4 bit



⑥ ALU:

combines a variety of arithmetic and logical operation in a single unit.

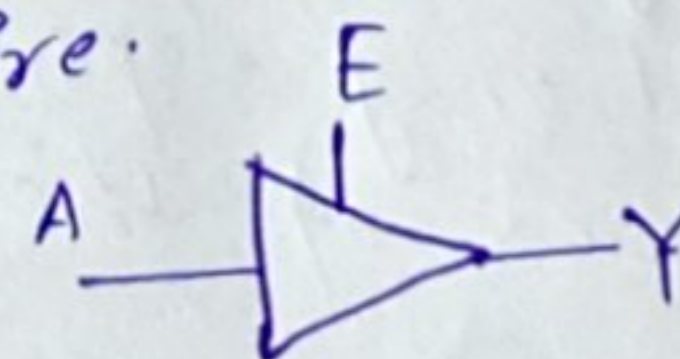


⑦ Tri-State Buffer:

Enables gating of different signals onto a wire.

Uses:

- Prevents collisions of value
- Connecting to the bus
- Making Multiplexers





## Implementation of Tri-State Buffer using CMOS:

### Logic Simplification:

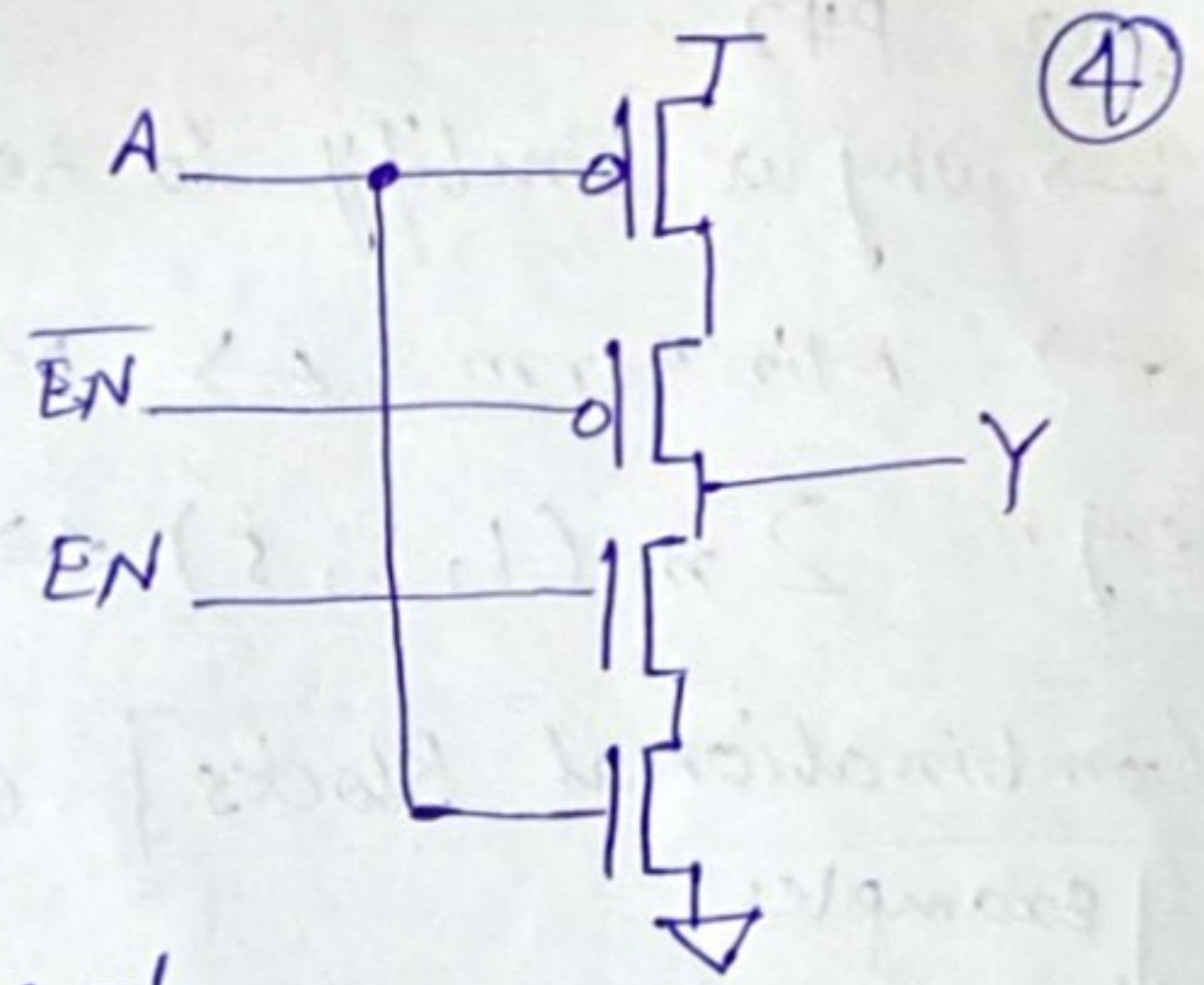
1. eliminate the expression, not making the difference.

e.g.

→ Priority circuit:

Inputs: "Requestors" with priority level.

Outputs: "Grant" signal for each request.



### 2. k-Maps

Sequential circuits: = combination of circuit + storage element.

① latches & FF > Static RAM > Dynamic RAM > other storage Tech  
[Cost] and [Speed]

→ ① SR Latch: uses two cross coupled inverters | NAND

$SR = 1 \rightarrow \text{idle}$

$S = 0, R = 1 \rightarrow Q = 1$

$R = 0, S = 1 \rightarrow Q = 0$

Advantage: Handles Metastability

### 1.1 Gated D-Latch

Uses a "write enable" signal to control when the input (D) is captured, for reliable storage.

### Register:

Parallel combination of D-latches for storing multiple bits.

### Memory:

→ combination of Registers. having unique address  
→ every memory location has a unique address

### Addressing Memory:

→ using Address decoder + Registers + Mux.

→ Data is written when write is enabled.

### Sequential Logic Circuits

They not only depend upon present input but also keep track of the past input.



## Asynchronous and Synchronous

(5)

- State transition occurs when they occurs
  - State transition takes place after fixed unit of time.
  - Difficult to Debug
  - Easy to Debug
- clock signal is used to (take) keep track of time.

### Finite State Machine :

→ pictorially show state and how transition occurs.

#### Elements

1. Finite States
2. Finite external input
3. Finite External output
4. specification of <sup>all</sup> state transitions
5. Specification of what determines each external value output.

#### Parts

1. next State Logic ,
2. state Register ,
3. output Logic.

Problems with D-latch:

→ It is transparent. Unwanted change propagates.

### D-Flip Flop :

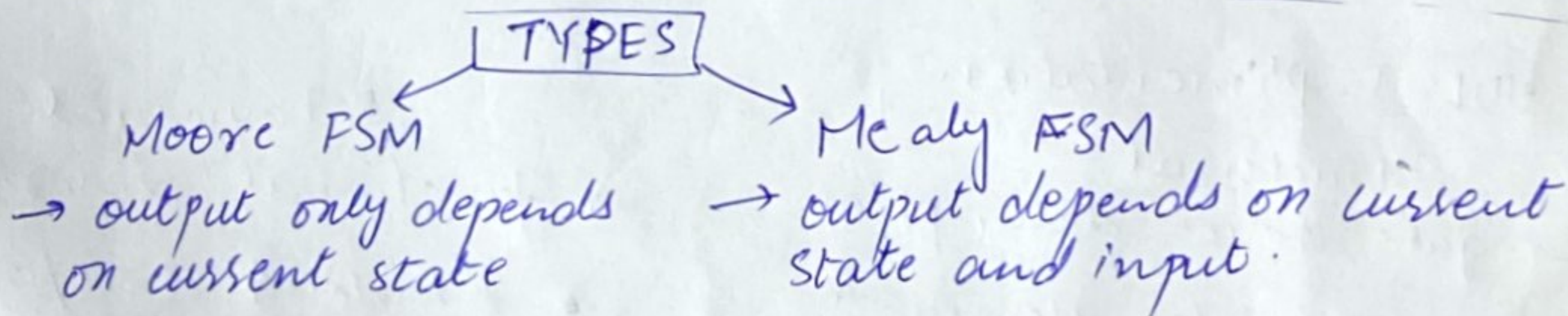
uses two back to back D-latches

- Resolves the issue of metastability.
- Edge triggered state element.

### D-Flip Flop Based Register:

(later)

FSMs:



Example:

"Smart" traffic light controller.

How to Solve Problem:

- ① Analyze
- ② state transition diagram
- ③ Truth table (Includes state Encoding)
- ④ Schematic (Timing Diagram for verification)



# FSM State Encoded

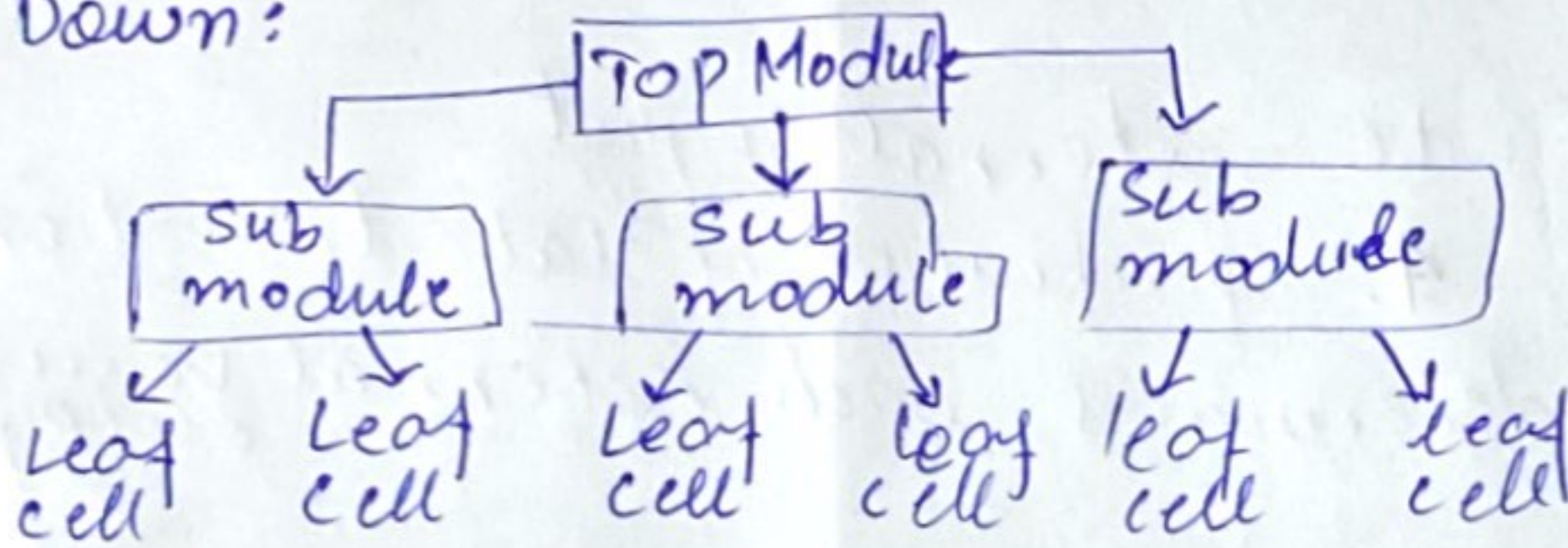
(6)

| Types: | 1. Fully Encoded                           | 2. 1-Hot Encoded                       | 3. Output Encoded              |
|--------|--------------------------------------------|----------------------------------------|--------------------------------|
|        | $\log_2(\text{num\_states})$ to represent. | uses num_states bit to represent state | output are directly accessible |

## Lec # 5

### Design Methodology

Top Down:



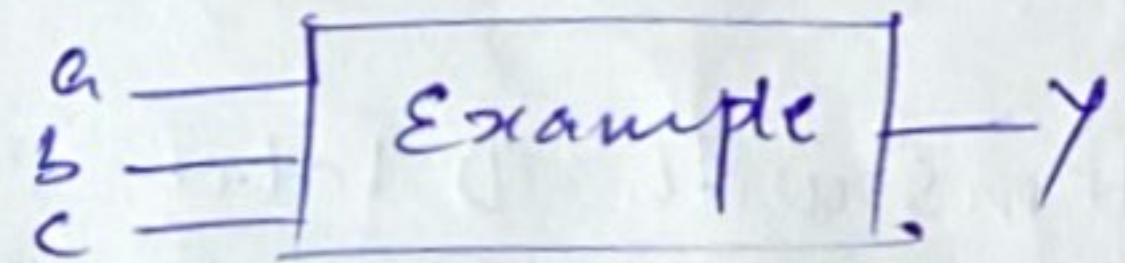
Bottom up:

opposite of Top down.

Defining module in verilog

```

module example (a, b, c, y);
    input a;
    input b;
    input c;
    output y;
end module
    
```



- To Define a multibit (say 4 bit) → [3:0]a
- Bit slicing e.g. a[3:1]
- Bit Concatandtion { a[2], a[0] }
- Bit Multiplication { 4{a[0]} }

## HDL Implimentation

### Structural

→ gate level description

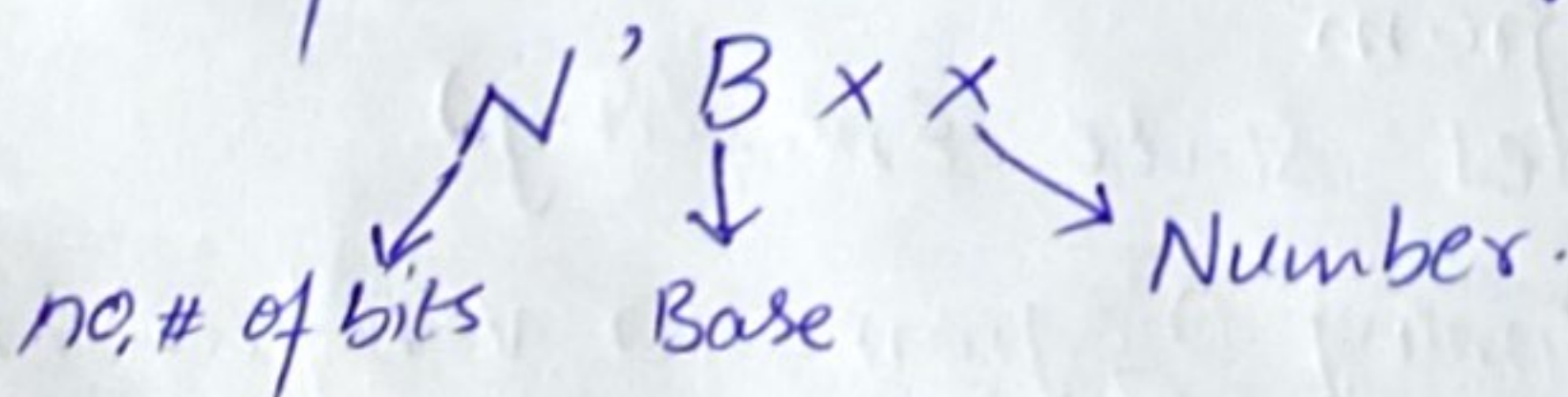
- Sub modules are made and connected via wires.

### Behavioural

→ functional description

- Behaviour is defined using the primitives using operators.
- Assign key word is used.

Number Representation is verilog:





HDL code  $\rightarrow$  Synthesis  $\rightarrow$  Simulation

(7)

Parameterized Modules:

# (parameter width = 8)

↑ width now will be replaced by 8.

### Good practices:

- $\rightarrow$  consistent naming
- $\rightarrow$  MSB to LSB ordering
- $\rightarrow$  one module per file
- $\rightarrow$  File name should match module name
- $\rightarrow$  Verilog describes behaviour.

### Sequential Logic in Verilog:

- $\rightarrow$  State transition is triggered by clk signal.
- $\rightarrow$  constructs like:  
always, posedge / negedge is used.

Sync and Async Reset.

↓  
depends on clock      ↓  
does not depend on clock.

- Sequential statements are within an always block.
- Sequential block is triggered with a change in sensitivity list.
- Signals assigned within an always must be declared as reg.
- Use non blocking assignments and don't use assign within always block.

### always Block

- $\rightarrow$  defines combinational logic if output is always updated
- $\rightarrow$  if, else, case can be used in always.

### Blocking Assignment.

- All assignments are made at the end of the block.
- All assignments are made in parallel, process flow is not blocked.

### Non Blocking Assignment

- Each assignment is made immediately
- Process waits until the first assignment is complete, it blocks the progress.



## Combinational Circuit Timing:

- we get delay in circuits due to:
- capacitance & resistance in circuit.
  - temperature
  - Aging of circuit.

Contamination Delay:  
delay until output starts changing

Propagation Delay  
delay until output finishes changing.

- longest path is critical path might cause longest delay

Output Glitches: one input transition causes multiple output transition.  
Incorrect triggering, (Unwanted Output).  
Cause: path timing difference.

## Sequential Circuit timing:

Timing Constraints in D Flip Flop:

- D must be stable when sampled.

Setup Time: time before clk edge that data must be stable.

Hold Time: time after clk edge that data must be stable.

Aperture Time: time around clk edge that must be stable.

$t_{cq} / t_{pcq}$  — clock-to-q delay.

$t_{cd} / t_{pd}$  — combinational logic delay.

$t_{setup}$  — time that FF inputs must be stable before next clock edge.

$t_{hold}$  — time that FF input must be stable after a clock edge.

$T_c$  — clock period.

## Clock Skew:

clocks too have delays.  
The time difference between two clock edges.



## Circuit Verification

⑨

→ test functionality, → test timing constraints  
All these tests are very time consuming.  
solution:

- ① check functionality at high level.
- ② check only timing power at low level.

### functional Verification:

- tests logical correctness.
- timing is ignored.  
delay
- using the test bench.
- logic is analysed.
- Simulation (Not Synthesis)

### Test Benches types:

| Test bench    | I/O Generation | Error Checking |
|---------------|----------------|----------------|
| Simple        | Manual         | Manual         |
| Self checking | Manual         | Auto           |
| Automatic     | Auto           | Auto           |

### Golden Model:

- Represent ideal circuit behaviours
- might be difficult to write.
- Used in Automatic testing.

### Timing Verification:

#### High level Simulation:

- timing is modeled using " #X "
- Not accurate as real circuits.

#### Circuit level Simulation:

- Need to synthesis the design first.
- design flow specific.